Практические задания по дисциплине «Виртуальные машины»

Общие требования к программной реализации работ

Для реализации необходимо использовать язык программирования Си, если в описании к лабораторной работе не сказано иное.

Исходный код лабораторных работ размещать в репозитории Gitlab факультета ПИиКТ.

Также необходимо следовать данным пунктам:

1. **«Нет» статичности.** Все структуры данных должны допускать создание множества их экземпляров.
2. **«Нет» «магическим» константам.** Все значения должны либо вычисляться из обрабатываемых программой данных, либо задаваться с помощью аргументов командной строки или конфигурационных файлов.
3. **«Нет» бесконечным циклам.** Все циклы должны иметь понятные условия выхода: не допускается использовать, например, while (true), for (; ;) и т.д.
4. **«Нет» утечке ресурсов.** Все ресурсы, которые были использованы в программе и требуют освобождения (закрытия), должны корректно освобождаться (закрываться) независимо от возникновения ошибочных ситуаций или исключений. Например, открытый файл должен быть закрыт после того, как он перестал использоваться в программе; аллоцированная вручную память обязательно должна освобождаться.
5. **«Нет» неожиданным завершениям программы.** Все процессы, нити (threads) должны корректно завершаться в результате выполнения работы, а не прерываться функциями вида Abort/Exit.
6. **«Нет» побайтовому вводу-выводу.** Все данные должны обрабатываться частями (блоками) известного размера, с учетом целесообразного размера буфера.

Настоятельно рекомендуется:

- В начале работы подготовить окружение разработчика, включающее отладчик, поддерживающий визуализацию структур данных и возможность отладки программ, выполняющихся под управлением ОС семейств Windows и *NIX.
- Сборку проекта осуществлять с помощью кроссплатформенных средств автоматизации, таких как мэйкфайлы.
- Следовать общим принципам грамотной разработки ПО, таким как SOLID, DRY, и др., грамотно использовать непрозрачные типы данных (opaque data types), разделять публичную и приватную функциональность модулей.

Оформление отчетов

По каждому из заданий должен быть представлен отчет, содержащий следующие части:

1. Титульный лист установленной формы, включающей следующие сведения: отчёт к практическому заданию №X по дисциплине У студента Z, группа, наименование факультета и вуза, год обучения.
2. Цели – описание цели задания (см. текст задания)
3. Задачи – путь достижения цели, что именно нужно было сделать для выполнения задания (план хода вашей работы)
4. Описание работы – внешнее описание созданной программы, состава модулей, способов её использования, примеры входной и выходной информации (модули, интерфейсы, тесты)
5. Аспекты реализации – внутреннее описание созданной программы, особенности алгоритмов, примеры кода
6. Результаты – что было сделано для выполнения задач кратко по пунктам (созданные артефакты, результаты тестов, количественные оценки)
7. Выводы – что было достигнуто в отношении цели задания. (что показали тесты, почему, как это было достигнуто, чему научились, качественные оценки)

Описание заданий

Задание 1

Используя комплекс программ, разработанный в первом семестре изучения дисциплины Системное программное обеспечение, реализовать формирование модели программы в терминах системы и модели памяти команд виртуальной машины по варианту. Полученную модель программы сериализовать и обеспечить выполнение программы посредством передачи сериализованного представления существующей реализации виртуальной машины.

Порядок выполнения:

1. Изучить особенности виртуальной машины по варианту.

1.1. Изучить соответствующие систему команд и модель памяти.

1.2. Изучить существующие реализации данной ВМ и поддерживаемые ими форматы представления программ.

1.3. Изучить внутреннюю организацию формата представления модели программ и особенности хода выполнения представленной таким образом программы.

2. Реализовать модуль, отвечающий за формирование модели программы в терминах целевой ВМ.

2.1. Описать необходимые структуры данных для представления модели программы в терминах целевой ВМ.

2.2. Программный интерфейс модуля принимает на вход структуру данных, содержащую графы потока управления и информацию о локальных переменных и сигнатурах для набора подпрограмм, а также информацию о необходимых пользовательских и встроенных типах данных.

2.3. В результате работы порождается структура данных, разработанная в п.2.1, в которой полученная в соответствии с п.2.2 информация представления в терминах целевой ВМ по мере необходимости.

3. Реализовать модуль, осуществляющий сериализацию сформированной модулем из п.2 структуры данных в выходной файл, организованный в соответствии с изученным форматом из п.1.3.

4. Реализовать основную программу, формирующую сериализованное представление в терминах целевой ВМ для подпрограмм, описанных файлах, имена которых переданы через аргументы командной строки.

4.1. Использовать модули, разработанные ранее в ходе изучения дисциплины Системное программное обеспечение для анализа входного текста.

4.2. Использовать модули, разработанные в п.2 и п.3 для формирования выходного файла, содержащего сериализованное представление программы.

5. Используя существующую реализацию ВМ, показать корректность формируемых выходных файлов и продемонстрировать выполнение тестовых программ.

6. Согласовать содержание отчёта с преподавателем, при необходимости внести изменения в разработанную программную систему и соответствующие правки в отчёт.

Задание 2

Дополнить разработанную в предшествующем задании программу поддержкой одной или нескольких высокоуровневых синтаксических конструкций по варианту. Поддержать корректное формирование выходных файлов и выполнение соответствующих программ.

Порядок выполнения:

1. Изучить внутреннюю организацию заданных синтаксических конструкций и соответствующую семантику.

2. Спроектировать набор необходимых паттернов для модели программы, описывающих изученную в п.1 семантику.

3. Добавить поддержку новых синтаксических конструкций в модуль, реализующий синтаксический анализ.

4. Поддержать формирование необходимого паттерна в модели программы (формирование дополнительных подпрограмм и пользовательских типов, изначально не описанных во входном тексте)

5. Используя существующую реализацию ВМ, показать корректность формируемых выходных файлов и продемонстрировать выполнение тестовых программ.

6. Согласовать содержание отчёта с преподавателем, при необходимости внести изменения в разработанную программную систему и соответствующие правки в отчёт.

Задание 3

Используя комплекс программ, разработанный в первом семестре изучения дисциплины Системное программное обеспечение, реализовать алгоритм динамического автоматического управления памятью для выходной программы, выполняющейся на учебной ВМ или в терминах аппаратной системы команд.

Порядок выполнения:

1. Изучить алгоритм динамического автоматического управления памятью по варианту.

2. Реализовать изученный алгоритм для учебной ВМ используя учебный входной язык и/или ассемблер учебной ВМ, при необходимости добавив вывод необходимой служебной информации при формировании линейного кода.

3. Написать тестовую программу, проверяющую корректность работы реализованного алгоритма управления памятью и запустить её на учебной ВМ. Собрать необходимую статистику и показать корректность работы реализованного алгоритма управления памятью.

4. Согласовать содержание отчёта с преподавателем, при необходимости внести изменения в разработанную программную систему и соответствующие правки в отчёт.

Задание 4

Используя язык высокого уровня по варианту, создать консольное или графическое приложение, использующее подпрограммы, реализованные с помощью комплекса программ, доработанного в ходе выполнения практического задания 1. При необходимости реализовать дополнительную библиотеку, обеспечивающую вызов функций, выполняющихся в рамках высокоуровневой ВМ, из программы на языке высокого уровня.

Порядок выполнения:

1. Изучить средства FFI высокоуровневого языка по варианту.

2. Изучить средства FFI высокоуровневой ВМ, использованной в ходе выполнения практического задания 1.

3. Разработать учебное приложение по варианту

3.1. Реализовать набор подпрограмм с помощью комплекса программ, доработанного в ходе выполнения практического задания 1.

3.2. Реализовать основную часть приложения на языке высокого уровня по варианту.

4. Согласовать содержание отчёта с преподавателем, при необходимости внести изменения в разработанную программную систему и соответствующие правки в отчёт.

Варианты к заданию 1

Вариант	**Высокоуровневая ВМ**
1	LLVM
2	CLR (ECMA-335)
3	BEAM
4	NekoVM
5	JVM
6	MoarVM

Варианты к заданию 2

Вариант	**Высокоуровневая конструкция**
1	async-await
2	foreach loop + generators
3	variant types + deconstructing match
4	first-order function type + local functions + closures
5	method reference + reference type + tailcall

Варианты к заданию 3

Вариант	**Способ автоматического освобождения памяти**
1	Умные указатели с учётом циклов
2	Полная сборка методом «пометить и освободить»
3	Полная сборка методом копирования выживших

Варианты к заданию 4

Вариант	**Язык высокого уровня**
1	C# (или другой .NET-совместимый язык по согласованию с преподавателем)
2	Java (Scala/Kotlin или другой JVM-совместимый язык по согласованию с преподавателем)
3	Python
4	JavaScript (или Typescript, другой язык по согласованию с преподавателем) + Node.js
5	PHP
6	Perl
7	Ruby
8	Erlang
9	Lua